

Setup and technical guide

A self-contained photobooth that runs on one Mac with no internet: a Canon camera, a touchscreen and a SELPHY printer, all attached to the same machine.

A guest taps a layout, poses for six photos with a countdown, taps their favourites, picks a frame design and a look, adds keychains or extra copies, and gets a pickup code. Staff take payment and print from their phone.

For running an event, give staff **OPERATORS-GUIDE.pdf**. This guide is for whoever sets the booth up or works on it.

The booth folder shows staff only what they need: `START-B00TH`, `CHECK-B00TH` and the two staff PDFs. Everything else is in `app/`, and every path in this guide is inside `app/`. The launchers `cd` into it, and so does anything run by hand: `cd app` first.

How it is built

No server, no database, no cloud. Photos go from the camera into the app, are composited on a canvas at true print size (1200 x 1800 = 4 x 6 in at 300 dpi) by the same code that draws the on-screen preview, and go to the printer. What the guest approves is exactly what prints.

<code>main.js</code> (main process)	<code>renderer/</code> (the touchscreen)
<code>camera-edsdk.js</code> → camera-helper (Canon EDSDK)	<code>app.js</code> screens, timing, compositing
<code>printer.js</code> → lp / CUPS	<code>sheets.js</code> print sheets, also run in a hidden worker window
<code>server.js</code> → the staff phone	<code>styles.css</code> , <code>index.html</code>
<code>queue.js</code> orders, on disk	
<code>supplies.js</code> paper and ink counts	
_____ preload.js bridge _____/	

The camera is driven through Canon's EDSDK by `camera-helper/`, a small Mac app with no window that holds one connection to the camera for the whole night. Live view, focus and capture all go through it, so the preview never drops between shots, focus happens during the countdown, and a photo lands about a second after zero. The EDSDK only works from inside a real Mac app with an event loop, which is why it is a separate helper and not part of Electron.

Orders are decoupled from printing. A finished session becomes an order in `sessions/queue.json` and the booth locks itself for the next group. Staff release prints from the phone once paid.

Requirements

- A Mac, Apple Silicon or Intel
- **Canon's EDSDK** in a folder called `EDSDK` inside `app/`. It is licensed from Canon and never committed: copy it across by hand.
- Apple's command-line tools, to build the camera helper: `xcode-select --install`
- Node.js 20 or newer (setup installs it)
- A Canon DSLR on USB. Developed against the EOS Rebel T6 / 1300D.
- A Canon SELPHY CP1500, added in System Settings > Printers & Scanners

Setting up a Mac

Double-click **START-BOOTH.command**. On a new Mac it installs Homebrew and Node.js, downloads the app's parts, builds the camera helper from `EDSDK/`, runs the self-test, then checks the camera and printer before opening the booth. It needs internet and an administrator's password the first time.

If the newest SDK in the command-line tools will not link (a half-updated install, or a macOS beta), the helper build falls back to an older SDK by itself, and says how to repair the tools if none works.

Moving to another Mac

```
scripts/make-bundle.sh      # -> ~/Desktop/Xinhmo-Booth.zip
```

The zip holds the committed files, the git history (so the other Mac can `git pull`) and `app/EDSDK`. Keep it private: it contains Canon's SDK and the staff code. On the other Mac, unzip it, clear the download flag, and double-click START-BOOTH:

```
xattr -dr com.apple.quarantine ~/Documents/kiosk
```

A Mac set up before the app moved into `app/` updates with a plain `git pull`. The next START-BOOTH or CHECK-BOOTH moves what git does not track (`EDSDK`, the downloaded parts, the built camera helper, photos, orders and logs) into `app/` by itself, so nothing is downloaded again or lost.

Updating from afar

Staff can put in a pushed fix without git: **EMERGENCY/UPDATE.command** runs `scripts/update.sh`, which sets the checkout to exactly `origin/master` (no merge, so nothing can block it), keeps every setting this Mac changed from what it was shipped with (`scripts/merge-settings.js`), sets any other stray edit aside in `git stash`, and downloads new parts only if `package-lock.json` changed. **UNDO-UPDATE.command** goes back to the version before, with no internet. Every message is in plain words for staff; **EMERGENCY.pdf** walks them through it.

So **anything pushed to master can reach the event Mac**. Push only what has been tested.

Running it

Command	Camera	Printing	Window
<code>START-B00TH.command</code>	real	real	full screen, checked first
<code>CHECK-B00TH.command</code>	checked	checked	just the check, no booth
<code>npm start</code>	real	real	full screen
<code>npm run windowed</code>	real	real	a normal window
<code>npm run dev</code>	real	off	a normal window
<code>npm run mock</code>	generated photos	off	a normal window

`npm run dev` never prints. For a real test, use `npm run windowed`.

Settings

Everything adjustable lives in `settings.json`. It is watched: most changes reach the running booth within a second. `staffPort` and `kiosk` need a restart. Defaults and comments for every setting are in `config.js`.

Setting	Meaning
<code>captureCount</code>	Photos per session
<code>countdownSeconds</code>	Countdown before each photo
<code>readySeconds</code>	The get-ready screen before the first photo
<code>pickSeconds</code> , <code>frameSeconds</code> , <code>filterSeconds</code>	Time on each choosing step
<code>cameraDriver</code>	<code>"edsdk"</code> (Canon SDK). <code>"gphoto2"</code> only if chosen on purpose; there is no automatic fallback
<code>cameraFocusLeadSeconds</code>	How long before the shutter focusing starts (4.5)
<code>printerName</code>	CUPS queue name, from <code>lpstat -p</code>
<code>printerMedia</code>	CUPS paper size, normally <code>Postcard(4x6in)</code>
<code>printDryRun</code>	<code>true</code> saves prints instead of printing
<code>mockCamera</code>	<code>true</code> uses generated photos

Setting	Meaning
<code>staffPin</code>	The staff code. Change it.
<code>staffPort</code>	Port of the staff phone page (8080)
<code>lockAfterSession</code>	Lock the booth after each group until staff start the next
<code>idleResetSeconds</code>	Give up on an abandoned session
<code>paperCassetteSheets</code> , <code>inkCassettePrints</code>	18 and 36 for the CP1500
<code>pricing</code>	Layout prices, extra copies, and add-ons (see Keychains)
<code>print.safeArea</code>	How much the printer trims at each edge, in mm

Environment variables override the file for one run: `PB MOCK_CAMERA` , `PB_PRINT_DRYRUN` , `PB_KIOSK` , `PB_CAMERA_DRIVER` , `PB_PRINTER_NAME` , and, for tests, `PB_SESSIONS_DIR` and `PB_SETTINGS_PATH` .

Camera

- **Lens switch on AF.** The booth focuses during the countdown, starting `cameraFocusLeadSeconds` before the shot; the 1300D's live-view focus takes about 3 seconds to lock.
- **Run it on the power adapter** (Canon ACK-E10 for the T6). The battery will not last an evening.
- Turn **auto power off** to Disable, and the camera's **Wi-Fi off** (it disables USB).
- Photos are sent straight to the Mac; a memory card is not needed.
- Quit Photos, Image Capture and EOS Utility. Do not kill macOS's `ptpcamerad` : the EDSDK reaches the camera through it.

Mark where people should stand, and light them: anyone who steps forward or back after focus locks comes out soft.

`CHECK-B00TH.command` opens the camera through the same helper the booth uses and needs a live picture back before it says OK.

Printer

Confirm printing works outside the app first:

```
lpstat -p -d
lp -d Canon_SELPHY_CP1500 -o "media=Postcard(4x6in)" -o fit-to-page some.jpg
```

The CP1500's paper tray holds 18 sheets and each ink cassette prints 36; a KP-108IN pack is 108 sheets and three cassettes. The booth counts sheets and warns staff before either runs out. The SELPHY itself

says nothing while idle, even with its tray out, but once a print is waiting it names what stopped it (`input-tray-missing` , `media-empty-error` , `marker-supply-empty-error`), CUPS repeats that in `lpstat -l -p` , and the phone shows *Tray out*, *Out of paper* or *Out of ink* in red. Staff tap Paper or Ink on the phone after reloading.

The calibration sheet (staff panel or phone) shows how much each edge loses. Put what you measure into `print.safeArea` .

Keychains and charms

Add-ons with a size are cut from the session's strip.

```
{ "id": "keychain", "price": 8, "heightMm": 76.2, "widthMm": 25.4,
  "perSheet": 4, "stripsPerItem": 2 }
```

A keychain insert is **3 x 1 in**, the strip's own 1:3 shape, so it fills the insert with nothing added. A keychain is two-sided, so it takes **two identical strips**, and four strips to a sheet is **two keychains**. Each order's strips go on sheets of their own, so nobody waits for strangers to fill a sheet. Sheets are drawn in a hidden worker window, not on the guest's screen.

Grand layouts are one large sheet with no strip, so they offer no keychains or charms; the menu, the order screen and the phone all say so.

Frames and designs

Layouts (Grand, Trio, Quad) are in `frames/frames.json` . The designer's frame artwork is in `frames/designs/` , imported from PNGs with transparent photo holes:

```
uv run --with pillow --with numpy --with scipy scripts/import-designs.py ~/Downloads
```

The importer finds each hole, works out the photo slots, and records which holes are shaped (hearts, ovals): those show the whole photo inside rather than cropping it. For editing words, colours and sizes on screen, see [docs/Editing-the-screens.pdf](#).

The staff phone

`server.js` serves a page on `staffPort` to any phone on the same network, behind the staff code. The address is given by the Mac's local hostname first (`http://Chaus-baby-.local:8080` on the event Mac), which stays the same on any network, then by number in case a phone cannot resolve the name. The hostname is under System Settings > General > Sharing > Local hostname. **Orders** lists waiting and finished orders, each with its print buttons. **Booth** shows the camera, its driver, the printer, paper, ink and storage, with controls to restart the camera, test print, end a session or shut down. **Settings** changes the timings. Settings can only be changed there or in `settings.json` , never by a guest.

The booth's own staff panel (hold the top-left corner for 2 seconds) has the same controls, for when no phone is available.

Staying alive

- The camera helper is restarted if it dies, reconnects when the camera comes back, and keeps the camera from sleeping.
- The order queue is written atomically, with a backup, and survives a crash or a full disk.
- A session that would not fit on the disk is refused at the welcome screen.
- If the screen process dies, the window reloads; if the app exits, START-BOOTH restarts it.
- A failed photo does not end the session.
- Logs are in `logs/`, one file a day.

Locking the screen down

macOS has no single-app lock. The strongest measure is physical: **unplug the keyboard during operation**. The app runs full screen, blocks quitting, and swallows the common shortcuts, but treat those as a deterrent. For unattended use, also create a dedicated macOS account with automatic login, add START-BOOTH as a login item, and turn off screen saver, display sleep and automatic updates.

Testing

```
npm run check      # self-test: settings, frames, queue, supplies, printer
node scripts/e2e.js # a whole session end to end, generated photos
```

`e2e.js` uses its own temporary sessions folder and settings file, so it never touches the booth's photos, orders or `settings.json`. It cannot run while the booth is open (one instance at a time). Both run on any Mac; the camera and printer are checked with `CHECK-BOOTH.command`.

Documents

`npm run docs` rebuilds the PDFs from their sources: this file and, in `docs/`, `START-HERE.md`, `OPERATORS-GUIDE.md`, `FLOW.md` and `UI-GUIDE.md`. The two staff PDFs are written to the top of the booth folder, the rest to `docs/`.